

Design patterns

COMPLETE LEARNING GUIDE

What You'll Learn

Design patterns are named, reusable solutions to recurring object-oriented design problems, and knowing them gives you a shared vocabulary with interviewers and teammates — saying "I'd use a Strategy here" communicates an entire structural approach in three words. This guide covers five of the patterns most frequently asked about and most genuinely useful in real Java codebases: Singleton (exactly one instance, with a careful look at thread-safe implementations), Factory (centralizing object creation logic), Builder (constructing complex objects step by step, avoiding telescoping constructors), Observer (one-to-many event notification), and Strategy (swapping an algorithm's implementation at runtime). Each is shown with working Java code and, just as importantly, a candid discussion of when NOT to reach for it — interviewers increasingly value candidates who recognize when a pattern would be over-engineering for the problem at hand.

1. Singleton: Exactly One Instance

Singleton guarantees a class has exactly one instance for the lifetime of the application and provides a single global access point to it — commonly used for configuration objects, logging, or a connection pool manager. The classic implementation problem is thread safety: a naive lazy-initialization check (if (instance == null) instance = new Singleton();) can race, with two threads both seeing null and both creating an instance. The enum-based singleton is widely considered the cleanest, most robust approach in Java, since the JVM guarantees enum instance creation is inherently thread-safe and serialization-safe with zero extra code.

JAVA

```
// Enum singleton -- thread-safe, serialization-safe, by construction
public enum ConfigManager {
    INSTANCE;

    private final Map<String, String> settings = new HashMap<>();

    public String get(String key) { return settings.get(key); }
}
// Usage: ConfigManager.INSTANCE.get("db.url")

// Double-checked locking, the classic alternative when a class (not enum) is
// preferred
public class LazySingleton {
    private static volatile LazySingleton instance;    // volatile is essential
    here

    public static LazySingleton getInstance() {
```

```
if (instance == null) {
```

```
        synchronized (LazySingleton.class) {
            if (instance == null) {           // check again inside the lock
                instance = new LazySingleton();
            }
        }
    }
    return instance;
}
}
```

2. Factory: Centralizing Object Creation

The Factory pattern moves the logic for deciding which concrete class to instantiate into a dedicated method or class, so calling code depends only on an interface or abstract type and never directly names a concrete implementation with `new`. This decouples client code from concrete classes — adding a new subtype later means changing only the factory, not every call site that creates one — and is especially valuable when construction requires conditional logic based on input (e.g. choosing a `PaymentProcessor` implementation based on a payment method string).

JAVA

```
interface Notification { void send(String message); }
class EmailNotification implements Notification { public void send(String m) { /
* ... */ } }
class SmsNotification implements Notification { public void send(String m) { /
* ... */ } }

class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "email" -> new EmailNotification();
            case "sms" -> new SmsNotification();
            default -> throw new IllegalArgumentException("Unknown type: " +
type); };
    }
}
// Caller never names EmailNotification or SmsNotification directly
Notification n = NotificationFactory.create("email");
```

3. Builder: Avoiding Telescoping Constructors

When a class has many optional fields, providing every combination as an overloaded constructor ("telescoping constructors") becomes unreadable and error-prone — callers can't tell which positional argument means what, and adding one more optional field multiplies the overload count. The Builder pattern instead uses a separate builder object with fluent, named setter-style methods returning `this`, culminating in a `build()` call that validates and constructs the final immutable object — Java's own `StringBuilder` and the widely-used Lombok `@Builder` annotation both embody this exact pattern.

JAVA

```
public class HttpRequest {
    private final String url;
    private final String method;
    private final Map<String, String> headers;

    private HttpRequest(Builder b) {
        this.url = b.url; this.method = b.method; this.headers = b.headers;
    }

    public static class Builder {
        private String url;
        private String method = "GET";
        private Map<String, String> headers = new HashMap<>();
    }
}
```

```
public Builder url(String url) { this.url = url; return this; }
```

```
        public Builder method(String method) { this.method = method; return
this; } public Builder header(String k, String v) { headers.put(k, v); return
this; } public HttpRequest build() { return new HttpRequest(this); }
    }
}
```

```
HttpRequest req = new HttpRequest.Builder()
    .url("https://api.example.com")
    .method("POST")
    .header("Content-Type", "application/json")
    .build();
```

4. Observer: One-to-Many Event Notification

The Observer pattern lets a "subject" object maintain a list of dependent "observer" objects and automatically notify all of them whenever its state changes, without the subject needing to know anything concrete about what each observer does with that notification — this is the foundation of GUI event listeners, pub/sub messaging systems, and reactive programming frameworks. The subject exposes subscribe()/unsubscribe() methods and an internal notify mechanism; observers implement a common interface (often just a single update() method or, in modern Java, a functional interface like Consumer<T>) so the subject can iterate and call each one without a type-specific switch.

JAVA

```
interface OrderObserver { void onOrderPlaced(Order order); }

class OrderService {
    private final List<OrderObserver> observers = new ArrayList<>();

    public void subscribe(OrderObserver o) { observers.add(o); }

    public void placeOrder(Order order) {
        // ... persist order ...
        for (OrderObserver o : observers) o.onOrderPlaced(order); // notify all
    }
}

// Independent observers: email confirmation, inventory update, analytics -- all
// decoupled
```

5. Strategy: Swapping Algorithms at Runtime

The Strategy pattern extracts an algorithm's variation into an interface, letting the concrete implementation be selected and swapped at runtime rather than hard-coded with a chain of if/else or switch statements scattered throughout the codebase. A common real-world example is a checkout system supporting multiple discount strategies (percentage off, flat amount off, buy-one-get-one) — each implements a shared DiscountStrategy interface, and the checkout code calls strategy.apply(cart) without caring which concrete strategy is currently plugged in. In modern Java, when a strategy is a single method, a lambda expression passed as a functional interface often replaces a full class hierarchy entirely.

JAVA

```
interface DiscountStrategy { double apply(double total); }

// Modern Java: lambdas as strategies, no separate classes needed
DiscountStrategy tenPercentOff = total -> total * 0.9;
DiscountStrategy flatFiveOff = total -> Math.max(0, total - 5.0);

double finalPrice = tenPercentOff.apply(cartTotal); // strategy selected at
// runtime
```

6. Knowing When NOT to Use a Pattern

Every one of these patterns adds a layer of indirection, which is a real cost in complexity and readability if the flexibility it buys is never actually needed — a Factory for a class with exactly one concrete implementation, or a Strategy interface with only one strategy ever written, is pure ceremony with no benefit. Interviewers increasingly probe for this judgment directly ("would you use a design pattern here?") because over-application of patterns ("pattern fever") is a real, common mistake among developers who've just learned them; the right answer is usually "not yet — introduce the abstraction when a second variant actually shows up, not preemptively."

Common Pitfalls

- Implementing a lazy Singleton without synchronization (or without volatile in double-checked locking), allowing two threads to race and create two separate instances.
- Reaching for a Factory, Strategy, or Builder for a class with no real variation or optional-field complexity, adding indirection with no actual benefit ("pattern fever").
- Forgetting volatile on a double-checked-locking Singleton's instance field, which can expose a partially-constructed object to another thread due to instruction reordering.
- Building a class with 6+ constructor parameters instead of switching to a Builder once telescoping constructors become unreadable.
- Letting an Observer's notify loop throw an unhandled exception from one observer, silently preventing every subsequent observer in the list from being notified.
- Confusing Strategy (swap an algorithm) with Factory (choose which object to construct) — they solve different problems and are often mistakenly used interchangeably in interview answers.

Key Takeaways

- Enum-based Singleton is the simplest thread-safe and serialization-safe way to guarantee exactly one instance in Java.
- Factory centralizes "which concrete class to instantiate" logic so client code depends only on an interface.
- Builder replaces telescoping constructors with fluent, named, step-by-step construction for classes with many optional fields.
- Observer lets a subject notify many decoupled listeners of a state change without knowing what each one does with it.
- Strategy extracts a swappable algorithm behind an interface — in modern Java, a single-method strategy is often just a lambda.
- Every pattern adds indirection — apply it when real variation exists or is clearly imminent, not preemptively.

Learn More

Refactoring Guru — Design Patterns

<https://refactoring.guru/design-patterns>

Baeldung — Singleton in Java

<https://www.baeldung.com/java-singleton>

Baeldung — Builder Pattern in Java

<https://www.baeldung.com/creational-design-patterns#builder>